Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

**Лабораторная работа №3**

По дисциплине: «Современные методы защиты компьютерных систем»

Тема: «Разработка модулей ядра операционной системы для динамического перехвата системных вызовов с использованием технологии eBPF»

Выполнил:
Студент 4 курса
Группа ИИ-26
Суховило В. С.
Проверила:
Хацкевич А. С.

Брест 2026

**Цель работы**

Практическое освоение разработки eBPF-зондов на C, перехвата системных вызовов Linux через tracepoints, фильтрации аномальных действий непосредственно в пространстве ядра и регистрации предупреждений в Python с использованием BCC (BPF Compiler Collection).

**Индивидуальное задание — вариант №4**

Номер варианта по журналу: (18 mod 15) + 1 = 4. Требуется отслеживать системный вызов sys_enter_ptrace и передавать событие в пространство пользователя только при запросе PTRACE_ATTACH — попытке подключения к другому процессу.

| **Параметр** | **Значение** |
|---|---|
| Tracepoint варианта | syscalls:sys_enter_ptrace |
| Событие-аномалия | request == PTRACE_ATTACH (16) |
| Фильтрация | До вызова perf_submit, в C-коде ядра |
| Передаваемые поля | PID/UID вызывающего процесса, target PID, request, comm |
| Файл аудита | logs/security_audit.log |

**Порядок выполнения**

Задание 1 — перехват execve и вывод PID, UID, comm. Задание 2 — перехват ptrace(PTRACE_ATTACH) с фильтрацией в ядре. Задание 3 — интеграция с Python и запись предупреждений. Задание 4 — сравнение Seccomp-BPF и eBPF, анализ риска TOCTOU.

**Условия проведения проверок**

Код предназначен для Ubuntu 22.04 / WSL2 с BCC и запуском от root. В доступной среде сборки BCC и tracefs отсутствуют, поэтому загрузить программу в ядро было невозможно. Выполнены отдельные настоящие проверки C-кода на стенде host mock, Python-обработчиков и локальной системной операции ptrace. Результаты этих проверок не приравниваются к наблюдению событий работающим eBPF-зондом.

**Задание 1. Базовый перехват запуска процессов**

Реализован tracepoint sys_enter_execve. В C-функции используется bpf_get_current_pid_tgid() для получения TGID, bpf_get_current_uid_gid() для UID и bpf_get_current_comm() для имени текущей задачи. Структура передаётся в perf-буфер execve_events.

```
#include <uapi/linux/ptrace.h>
#include <linux/sched.h>

/* Событие запуска процесса для передачи из ядра в Python. */
struct execve_event_t {
    u32 pid;
    u32 uid;
    char comm[TASK_COMM_LEN];
};

BPF_PERF_OUTPUT(execve_events);

TRACEPOINT_PROBE(syscalls, sys_enter_execve)
{
    struct execve_event_t event = {};
    event.pid = (u32)(bpf_get_current_pid_tgid() >> 32);
    event.uid = (u32)(bpf_get_current_uid_gid() & 0xffffffffULL);
    bpf_get_current_comm(&event.comm, sizeof(event.comm));
    execve_events.perf_submit(args, &event, sizeof(event));
    return 0;
}
```

Часть пользовательского пространства: скрипт main.py выбирает файл task1_execve.c, компилирует его посредством BPF(text=...), регистрирует обратный вызов для буфера execve_events и выполняет цикл perf_buffer_poll. Сведения о событии выводятся обработчиком monitor.handle_execve_event.

```
# Запуск в Ubuntu/WSL2 с BCC:
sudo /usr/bin/python3 main.py --task 1

# Во втором терминале:
/bin/ls -l
/bin/cat /etc/os-release
```

Особенность tracepoint на входе execve: поле comm может отражать старое имя процесса до замены исполняемого образа. Для точного имени нового файла следует отдельно считывать аргумент filename либо наблюдать событие sched_process_exec.

**Задание 2. Динамический анализ аномального поведения**

По варианту №4 требуется обнаруживать попытки ptrace(PTRACE_ATTACH). В отличие от наблюдения всех вызовов ptrace, фильтр выполняется непосредственно в kernel space до формирования сообщения. Это предотвращает передачу шумных событий с другими типами запроса.

```
#include <uapi/linux/ptrace.h>
#include <linux/sched.h>

/* Значение запроса PTRACE_ATTACH в Linux (uapi asm-generic/ptrace.h). */
#ifndef PTRACE_ATTACH
#define PTRACE_ATTACH 16
#endif

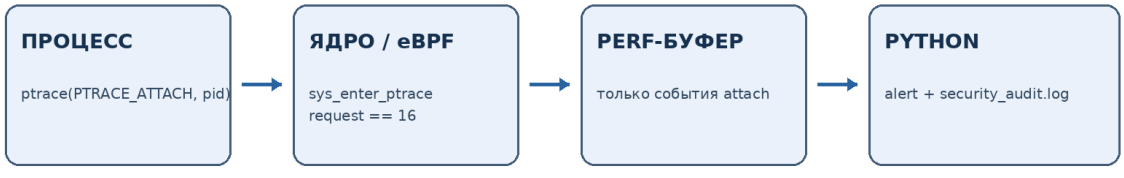
struct ptrace_event_t {
    u32 caller_pid;       /* TGID инициатора вызова ptrace */
    u32 caller_uid;       /* UID инициатора */
    s32 target_pid;       /* PID процесса-цели */
    u32 request;          /* тип ptrace-запроса */
    char comm[TASK_COMM_LEN];
};

BPF_PERF_OUTPUT(ptrace_events);

TRACEPOINT_PROBE(syscalls, sys_enter_ptrace)
{
    /* ФИЛЬТРАЦИЯ В ЯДРЕ: другие ptrace-запросы не передаются в Python. */
    if (args->request != PTRACE_ATTACH) {
        return 0;
    }

    /* Событие формируется только для запроса PTRACE_ATTACH. */
    struct ptrace_event_t event = {};
    event.caller_pid = (u32)(bpf_get_current_pid_tgid() >> 32);
    event.caller_uid = (u32)(bpf_get_current_uid_gid() & 0xffffffffULL);
    event.target_pid = (s32)args->pid;
    event.request = (u32)args->request;
    bpf_get_current_comm(&event.comm, sizeof(event.comm));
    ptrace_events.perf_submit(args, &event, sizeof(event));
    return 0;
}
```

Первая проверка в функции — if (args->request != PTRACE_ATTACH) return 0. Только после её прохождения C-зонд формирует объект ptrace_event_t и выполняет ptrace_events.perf_submit(...). Это выполняет обязательное требование методички о фильтрации до передачи в Python.




Фильтрация выполняется до передачи события из ядра; на входе tracepoint фиксируется попытка, не её исход.

*Рисунок 1 — Архитектура фильтрации и передачи событий (схема)*

**Проверка условий на уровне C**

Исходная функция TRACEPOINT_PROBE скомпилирована GCC в тестовом окружении со стабами BCC helper-функций. Проверены запросы PTRACE_TRACEME (0), PTRACE_SEIZE (16902), PTRACE_ATTACH (16). Событие создано только для PTRACE_ATTACH. Это проверка алгоритма, а не компиляция загрузчиком BCC и не проверка eBPF verifier.

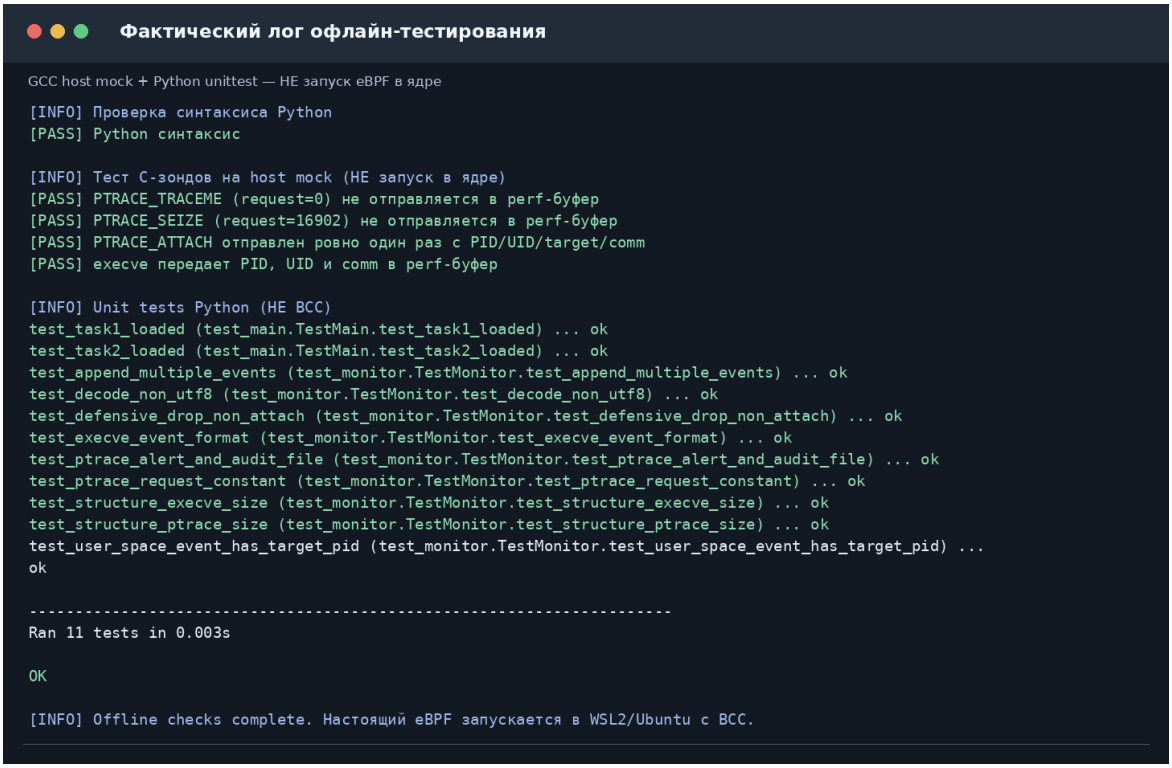



*Рисунок 2 — Фактический сохранённый лог offline-проверки C и Python (без загрузки eBPF)*

**Безопасное воспроизведение аномалии**

Для проверки создан скрипт scripts/generate_ptrace_attach.py. Он создаёт свой дочерний процесс, выполняет ptrace(PTRACE_ATTACH, child_pid), ожидает остановки, выполняет PTRACE_DETACH и завершает дочерний процесс. Тест не подключается к системным или чужим процессам.

```
# Отдельный терминал, когда монитор запущен:
/usr/bin/python3 scripts/generate_ptrace_attach.py
```

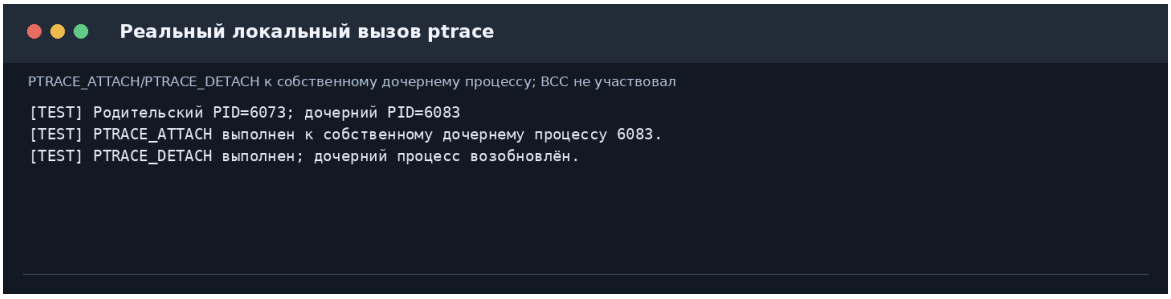


*Рисунок 3 — Реальный локальный вызов PTRACE_ATTACH/PTRACE_DETACH (без подключённого BCC)*

На этапе подготовки локальная попытка PTRACE_ATTACH к дочернему процессу и последующее отключение выполнены успешно. Однако свидетельство срабатывания kernel tracepoint отсутствует: его нужно получить в Ubuntu/WSL2 с поддержкой BCC.

**Задание 3. Интеграция с Python и аудит**

Обработчик в monitor.py получает данные из perf-буфера, переводит структуру ctypes в читаемый формат, добавляет временную метку и сохраняет одну строку в security_audit.log. Запись содержит caller PID, UID, target PID, request и comm.

```
class PtraceEvent(ctypes.Structure):
    _fields_ = [
        ("caller_pid", ctypes.c_uint32),
        ("caller_uid", ctypes.c_uint32),
        ("target_pid", ctypes.c_int32),
        ("request", ctypes.c_uint32),
        ("comm", ctypes.c_char * 16),
    ]

def handle_ptrace_event(_cpu, data, _size):
    event = ctypes.cast(data, ctypes.POINTER(PtraceEvent)).contents
    if event.request != PTRACE_ATTACH:
        return
    line = format_ptrace_event(event, timestamp())
    print(line, flush=True)
    LOG_FILE.parent.mkdir(parents=True, exist_ok=True)
    with LOG_FILE.open("a", encoding="utf-8") as stream:
        stream.write(line + "\n")
```

Функциональность обработчика проверена с искусственно созданным ctypes-событием (caller PID=6102, target PID=6103). Это демонстрирует формат вывода и запись файла, но не подтверждает поступление события из ядра.

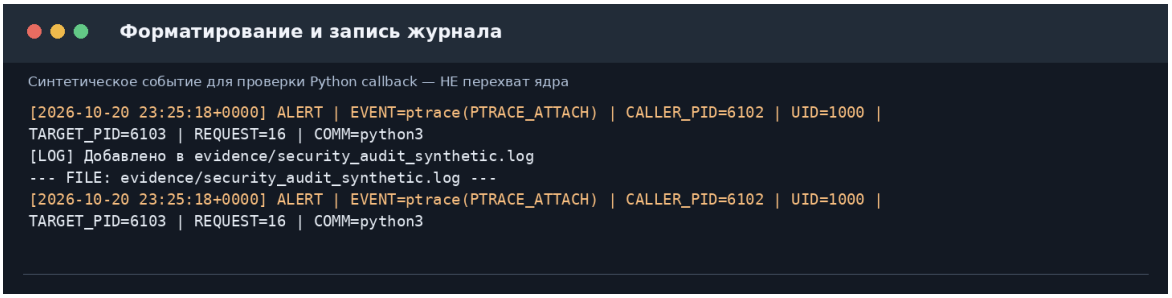



*Рисунок 4 — Проверка записи журнала на синтетическом событии (не live-eBPF)*

В реальной Ubuntu запись появится в logs/security_audit.log после вызова тестового PTRACE_ATTACH при активном Python-мониторе. Для отчёта преподавателю необходимо заменить синтетический пример подтверждённым скриншотом настоящего файла и монитора.

## Задание 4. Проектирование профилей защиты

### 4.1. Seccomp-BPF и eBPF tracepoints/kprobes

| Признак | Seccomp-BPF | eBPF tracepoint / kprobe |
|---|---|---|
| Основная роль | Ограничение доступных syscall | Наблюдение и аудит событий |
| Точка действия | До выполнения syscall; решение допустить/запретить | В точке трассировки без штатной блокировки syscall |
| Результат | Может вернуть отказ, отправить сигнал, применить действия фильтра | Отправляет сведения в perf-буфер |
| Типовые задачи | Песочницы, ограничения контейнеров и процессов | Мониторинг активности, диагностика, выявление аномалий |
| Применение к ptrace | Можно ограничить отдельные запросы, в т. ч. PTRACE_ATTACH | Фиксируется попытка PTRACE_ATTACH, независимо от итогового разрешения |

Seccomp-BPF оценивает значения аргументов системного вызова до исполнения и может запретить операцию. Программа на tracepoint/kprobe предназначена преимущественно для регистрации. Попытка изменять состояние ядра или произвольно прерывать выполнение из kprobe не является корректным способом контроля доступа и может привести к ошибкам и нестабильности. Для принудительного разрешения или отказа используют предназначенные для этого механизмы, например Seccomp или LSM.

### 4.2. Риск TOCTOU в контексте варианта №4

TOCTOU (Time-of-Check to Time-of-Use) возникает, если данные проверяются в один момент, а используются позже, когда они могли измениться. Особенно это актуально для указателей на пользовательскую память: другой поток может поменять строку или структуру между чтением пробой и использованием ядром.

В нашем зонде request и pid — скалярные аргументы уже вошедшего системного вызова, а не строки по пользовательским указателям. Поэтому типичный сценарий замены текста между проверкой и использованием к этим двум полям непосредственно не относится.

Утверждать, что другой поток изменит pid в ядре после регистрации события, было бы некорректно.

Если бы программа дополнительно исследовала данные по пользовательскому адресу (например, строку в другом syscall), следовало бы выполнить ограниченное по размеру безопасное копирование через bpf_probe_read_user_str() либо подходящий bpf_probe_read_user(), проверять возвращённый статус и работать с локальной неизменяемой копией. Это даёт согласованный снимок для анализа, но не гарантирует, что ядро позже использует те же байты. Для блокирования действий требуются проверки непосредственно в точке принятия решения, например LSM.

Зонд на sys_enter_ptrace отражает намерение вызвать PTRACE_ATTACH, а не факт успешного присоединения. Возможна блокировка со стороны прав доступа Linux, Yama или другого механизма. Для различения успешного и отклонённого действия требуется дополнительно обрабатывать sys_exit_ptrace и анализировать возвращаемый код.

**Результаты проверки и соответствие методичке**

| **Раздел** | **Фактически проверено** | **Для окончательной сдачи** |
|---|---|---|
| Задание 1 | C-функция execve в host mock; Python callback unit test | Скриншот live-монитора execve после ls/cat |
| Задание 2 | Фильтрация C для attach; реальный локальный ptrace к своему child | Live-алерт из sys_enter_ptrace в Ubuntu/WSL2 |
| Задание 3 | Формат и запись synthetic аудита в unit test | Реальный security_audit.log с событием из ядра |
| Задание 4 | Сравнение Seccomp/eBPF и анализ TOCTOU | Теоретическая часть завершена |

**Команды для окончательной демонстрации**

```
# Ubuntu/WSL2. Терминал A:
sudo /usr/bin/python3 main.py --task 1
# Терминал B: /bin/ls -l ; /bin/cat /etc/os-release

# Затем терминал A:
sudo /usr/bin/python3 main.py --task 2
# Терминал B:
/usr/bin/python3 scripts/generate_ptrace_attach.py
# Терминал C:
cat logs/security_audit.log
```

При отсутствии поддержки BCC в WSL2 следует использовать полноценную виртуальную машину Ubuntu. Изображения выполненных здесь offline-тестов оставлены в отчёте с достоверными подписями; обязательные live-eBPF-снимки пока не получены.

**Вывод:** Разработаны два C-зонда и Python-монитор для execve и ptrace(PTRACE_ATTACH) по варианту №4, фильтрация на стороне ядра, предупреждения и файловый аудит. Подтверждены 4 проверки логики C, 11 тестов Python и реальный PTRACE_ATTACH к собственному дочернему процессу. Загрузка eBPF в ядро и получение live-скриншотов остаются необходимым этапом проверки в Ubuntu/WSL2 с BCC.